

GPUMODE Lecture Study Notes: Lecture 5: Going Further with CUDA for Python Programmers

Contents

1 Core Problem the Lecture Addresses

The lecture addresses the next major step after learning the CUDA execution model: using the GPU memory hierarchy effectively. In the previous CUDA-for-Python lecture, the central idea was that a GPU can run many thousands of threads concurrently. That lets us expose parallelism for elementwise operations and naive matrix multiplication. However, raw parallelism alone is not enough. A kernel can launch many threads and still run slowly if every thread repeatedly loads data from high-latency global memory.

The main technical problem is therefore:

How do we reduce expensive global memory traffic by staging reusable data in shared memory?

The lecture uses matrix multiplication as the motivating example. A naive matrix multiplication kernel computes each output element $P_{r,c}$ by taking a dot product between row r of matrix M and column c of matrix N :

$$P_{r,c} = \sum_{i=0}^{K-1} M_{r,i} N_{i,c}.$$

In the naive version, each thread computes one output element and repeatedly reads elements of M and N from global memory. This is simple, but it wastes bandwidth because nearby output elements reuse many of the same input values. For example, two adjacent output columns $P_{r,c}$ and $P_{r,c+1}$ use the same row of M . Similarly, two adjacent output rows $P_{r,c}$ and $P_{r+1,c}$ use the same column of N . If each thread independently fetches everything from global memory, the kernel ignores this reuse.

The solution introduced in the lecture is **tiling**. Instead of computing each output element by reading entire rows and columns from global memory, each thread block cooperatively loads a tile of M and a tile of N into shared memory. Then the block computes a tile of the output using fast shared-memory loads.

The key memory hierarchy idea is:

Global memory is visible to all threads but is relatively slow. Shared memory is much faster, but it is only visible to threads within the same thread block. Therefore, if a group of threads needs to reuse the same data, arrange those threads into one block and stage the reused data in shared memory.

The lecture builds this idea progressively:

1. Review naive matrix multiplication in Python and CUDA.
2. Explain why repeated global memory reads are expensive.
3. Introduce tiles as small submatrices.
4. Write a Python simulation of shared memory.
5. Refactor the Python simulation to resemble CUDA.
6. Use Python threading and barriers to model CUDA thread synchronization.
7. Convert the logic to CUDA C with dynamic shared memory.

8. Compare dynamic shared memory with statically sized shared memory.
9. Explain why compile-time constants matter for optimization.
10. Introduce Numba CUDA as a Python-native CUDA development path.
11. Discuss CUDA simulation, debugging, compilation speed, Triton, and practical development workflow.

2 Required Background

2.1 CUDA Execution Model

CUDA organizes parallel work using a hierarchy:

grid $\rightarrow$ thread blocks $\rightarrow$ threads.

A **thread** executes one instance of a kernel. A **thread block** is a group of threads that can cooperate through shared memory and synchronization. A **grid** is a collection of blocks launched for a kernel.

For a two-dimensional problem such as matrix multiplication, it is natural to use a two-dimensional grid and two-dimensional blocks:

`blockIdx.x, blockIdx.y` identify the output tile,

`threadIdx.x, threadIdx.y` identify a thread inside the tile.

If the block has dimensions:

$$\text{blockDim.x} = T, \quad \text{blockDim.y} = T,$$

then a $T \times T$ block has:

$$T^2$$

threads. In the lecture, a common tile width is:

$$T = 16.$$

Thus each block contains:

$$16 \times 16 = 256$$

threads, and each block computes a 16×16 tile of the output matrix.

2.2 Global Memory

Global memory is the main GPU memory. In PyTorch, when we call:

```
x = x.cuda()
```

the tensor data is placed in GPU global memory. Global memory has high bandwidth compared with CPU memory, but from the perspective of an individual thread it is still much slower than on-chip memory. Accessing global memory repeatedly inside an inner loop can dominate runtime.

In the naive matrix multiplication kernel, every output element requires K loads from M and K loads from N :

$$\text{global loads per output element} = 2K.$$

For an $H \times K$ matrix multiplied by a $K \times W$ matrix, the output has HW elements, so the total number of scalar global loads in the naive conceptual model is:

$$2HWK.$$

This is a large amount of traffic, especially because many of those values are reused by neighboring threads.

2.3 Shared Memory

Shared memory is an on-chip memory region associated with a thread block. It is much faster than global memory. In the lecture, shared memory is described as roughly an order of magnitude faster than global memory. The exact ratio depends on hardware and access pattern, but the practical lesson is stable:

Use shared memory when multiple threads in a block reuse the same data.

Shared memory is not visible across blocks. If block 0 loads data into shared memory, block 1 cannot see it. This restriction shapes the algorithm: each block should compute a coherent region of the output that benefits from the same loaded input tiles.

2.4 Matrix Multiplication Notation

Let:

$$M \in \mathbb{R}^{H \times K}, \quad N \in \mathbb{R}^{K \times W}, \quad P \in \mathbb{R}^{H \times W}.$$

Then:

$$P = MN,$$

and each output element is:

$$P_{r,c} = \sum_{i=0}^{K-1} M_{r,i} N_{i,c}.$$

The row index is r , the column index is c , and the reduction index is i .

For a flattened row-major matrix, the address calculations are:

$$M_{r,i} \rightarrow M[rK + i],$$

$$N_{i,c} \rightarrow N[iW + c],$$

$$P_{r,c} \rightarrow P[rW + c].$$

These indexing formulas are essential when converting the Python logic into CUDA C, because kernels often receive flattened pointers.

2.5 Ceiling Division

When matrix dimensions are not exact multiples of the tile width, we need enough blocks to cover the whole matrix. This uses ceiling division:

$$\text{cdiv}(a, b) = \left\lceil \frac{a}{b} \right\rceil = \frac{a + b - 1}{b} \quad \text{with integer division.}$$

In Python:

```
def cdiv(a, b):
    return (a + b - 1) // b
```

For matrix multiplication with tile width T , the number of output tile rows is:

$$\left\lceil \frac{H}{T} \right\rceil,$$

and the number of output tile columns is:

$$\left\lceil \frac{W}{T} \right\rceil.$$

The number of phases along the reduction dimension is:

$$\left\lceil \frac{K}{T} \right\rceil.$$

3 Main Concepts

3.1 From Thread Parallelism to Memory Reuse

The first CUDA lesson focused on the fact that GPUs can run many threads. This lecture emphasizes that high throughput also depends on moving data efficiently. A matrix multiplication kernel performs many arithmetic operations, but those operations are only fast if the data is available close to the arithmetic units.

The naive approach is:

1. Assign one thread to one output element.
2. Compute the row and column for that output.
3. Loop over the reduction dimension K .

4. Load one element from M and one element from N from global memory.
5. Accumulate their product.
6. Store the final result.

This exposes enough parallelism, but it does not exploit locality. Consider computing a $T \times T$ output tile. The threads in that block need:

- T rows from M for a slice of the reduction dimension.
- T columns from N for the same slice of the reduction dimension.

Instead of each thread loading the needed values independently, the block can cooperatively load two $T \times T$ tiles:

$$M_{\text{tile}} \in \mathbb{R}^{T \times T}, \quad N_{\text{tile}} \in \mathbb{R}^{T \times T}.$$

Then every thread in the block reuses these shared-memory tiles to update one element of the output tile.

3.2 Tiling Matrix Multiplication

The tiled formula decomposes the reduction dimension into phases. Let T be the tile width. Define the number of phases:

$$P_{\text{phases}} = \left\lceil \frac{K}{T} \right\rceil.$$

For each output element $P_{r,c}$:

$$P_{r,c} = \sum_{\phi=0}^{P_{\text{phases}}-1} \sum_{j=0}^{T-1} M_{r,\phi T+j} N_{\phi T+j,c}.$$

When $\phi T + j \geq K$, the access is out of bounds and should be treated as zero padding:

$$M_{r,\phi T+j} = 0 \quad \text{if} \quad \phi T + j \geq K,$$

$$N_{\phi T+j,c} = 0 \quad \text{if} \quad \phi T + j \geq K.$$

This zero-padding rule lets every phase use the same inner loop length T , even for the final partial tile.

The block-level computation is:

1. Choose one output tile using `blockIdx.x` and `blockIdx.y`.
2. Each thread chooses one element inside that output tile using `threadIdx.x` and `threadIdx.y`.
3. For each reduction phase ϕ :
 - (a) Load one element of M_{tile} into shared memory.
 - (b) Load one element of N_{tile} into shared memory.

- (c) Synchronize all threads.
 - (d) Accumulate a length- T dot product from shared memory.
 - (e) Synchronize all threads again before overwriting shared memory in the next phase.
4. Write the accumulated result to global memory.

3.3 Why Tiling Reduces Global Memory Traffic

For a $T \times T$ output tile, there are T^2 output elements. In the naive approach, each output element reads T elements from M and T elements from N for each phase. Therefore, per phase, the naive block-level load count is:

$$2T^3.$$

In the tiled shared-memory approach, the block loads one $T \times T$ tile from M and one $T \times T$ tile from N :

$$2T^2$$

global memory loads per phase.

The reduction factor is:

$$\frac{2T^3}{2T^2} = T.$$

Thus, in the idealized model, tiling reduces global memory reads by a factor of approximately T . For $T = 16$:

$$\text{ideal global-memory load reduction} = 16 \times .$$

For $T = 32$:

$$\text{ideal global-memory load reduction} = 32 \times .$$

This does not automatically mean the whole kernel becomes $16\times$ or $32\times$ faster, because runtime also depends on synchronization overhead, occupancy, instruction scheduling, memory coalescing, register pressure, shared-memory bank conflicts, and compiler optimization. But it explains why tiling is the fundamental optimization for matrix multiplication.

3.4 Blocks as Output Tiles

The lecture emphasizes that a CUDA block is a programming-model concept, while a tile is an algorithmic concept. We choose to map them together:

$$\text{one CUDA block} \longleftrightarrow \text{one output tile}.$$

This mapping is powerful because all threads in one block can access the same shared memory. Therefore, the semantic tile of the output corresponds to the hardware cooperation group.

If:

$$T = \text{blockDim.x} = \text{blockDim.y},$$

then:

`blockIdx.y` selects the tile row,

`blockIdx.x` selects the tile column.

Inside the tile:

`threadIdx.y` selects the local row,

`threadIdx.x` selects the local column.

The global output coordinates are:

$$r = \text{blockIdx.y} \cdot T + \text{threadIdx.y},$$

$$c = \text{blockIdx.x} \cdot T + \text{threadIdx.x}.$$

3.5 Phases Along the Reduction Dimension

Each block computes a $T \times T$ tile of P . To do so, it moves across the K dimension in chunks of size T .

For phase ϕ , the starting reduction index is:

$$\text{idx} = \phi T.$$

The element loaded into shared memory for M by thread (t_y, t_x) is:

$$M_{r, \phi T + t_x}.$$

The element loaded into shared memory for N by thread (t_y, t_x) is:

$$N_{\phi T + t_y, c}.$$

The shared-memory arrays are:

$$M_s[t_y, t_x] = M_{r, \phi T + t_x},$$

$$N_s[t_y, t_x] = N_{\phi T + t_y, c}.$$

Then the same thread updates its accumulator:

$$\text{acc} \leftarrow \text{acc} + \sum_{j=0}^{T-1} M_s[t_y, j] N_s[j, t_x].$$

This is the core tiled matrix multiplication pattern.

3.6 Synchronization

Shared memory is only correct if every thread agrees on when a tile has been fully loaded and when it is safe to overwrite it. CUDA provides:

```
__syncthreads();
```

This is a block-level barrier. It means:

No thread in the block may pass this point until all threads in the block have reached it.

The tiled kernel needs two synchronization points per phase:

1. After loading M_s and N_s , before using them.
2. After using M_s and N_s , before overwriting them for the next phase.

Without the first barrier, some threads might start computing before other threads have finished loading shared memory. Without the second barrier, some threads might begin loading the next phase and overwrite shared memory while other threads are still using the current phase.

4 Hardware-Level Explanation

4.1 Streaming Multiprocessors and Blocks

A GPU contains multiple streaming multiprocessors, usually abbreviated as SMs. Each SM executes one or more resident thread blocks. Threads within a block are scheduled in groups called warps. On NVIDIA GPUs, a warp contains:

32

threads.

A CUDA block is scheduled onto one SM. Because all threads in a block reside on the same SM, they can communicate through shared memory. Threads in different blocks may run on different SMs and cannot directly synchronize or share shared memory.

This is why shared memory has block scope:

shared memory visibility = threads in the same block only.

4.2 Memory Hierarchy

A simplified GPU memory hierarchy is:

1. Registers: private to each thread, fastest, limited capacity.
2. Shared memory: visible to a block, very fast, on-chip, programmer-managed.
3. L1 cache: on-chip cache, behavior depends on architecture and access pattern.
4. L2 cache: shared across SMs.
5. Global memory: large off-chip memory, high bandwidth but high latency.

For tiled matrix multiplication, the accumulator usually lives in a register:

$$\text{acc} \in \text{per-thread registers.}$$

The input tiles live in shared memory:

$$M_s, N_s \in \text{per-block shared memory.}$$

The full matrices live in global memory:

$$M, N, P \in \text{global memory.}$$

The high-level flow is:

$$\text{global memory} \rightarrow \text{shared memory} \rightarrow \text{register accumulator} \rightarrow \text{global memory output.}$$

Visual Concept

Draw global memory at the top containing matrices M and N . Draw an arrow into one SM. Inside the SM, draw a thread block, two shared-memory tiles labeled M_s and N_s , and many threads each holding a private accumulator. Then draw the final write back to output matrix P in global memory.

4.3 Why Shared Memory Helps

Suppose a block computes a 16×16 output tile. That is 256 threads. For one phase, the block needs:

$$16 \times 16 = 256$$

elements from M , and:

$$16 \times 16 = 256$$

elements from N . The total is:

$$512$$

global loads per phase.

After loading these 512 elements, the block performs:

$$16 \times 16 \times 16 = 4096$$

multiply-add steps for that phase. Thus, each loaded element is reused by multiple threads.

In contrast, the naive approach would conceptually load:

$$2 \times 16 \times 16 \times 16 = 8192$$

input elements from global memory per phase.

So shared memory converts repeated global-memory loads into repeated shared-memory loads, increasing arithmetic intensity.

4.4 Arithmetic Intensity

Arithmetic intensity is:

$$\text{Arithmetic Intensity} = \frac{\text{FLOPs}}{\text{Bytes moved from slower memory}}.$$

For one $T \times T$ tile phase, the operation count is approximately:

$$2T^3$$

floating-point operations if we count one multiply and one add as two FLOPs. The shared-memory tiled global load volume is:

$$2T^2 \cdot \text{sizeof}(\text{float}).$$

For FP32:

$$\text{sizeof}(\text{float}) = 4 \text{ bytes.}$$

Therefore the global-memory arithmetic intensity per phase is:

$$\text{AI}_{\text{tiled}} = \frac{2T^3}{2T^2 \cdot 4} = \frac{T}{4} \text{ FLOPs per byte.}$$

For $T = 16$:

$$\text{AI}_{\text{tiled}} = \frac{16}{4} = 4 \text{ FLOPs per byte.}$$

For $T = 32$:

$$\text{AI}_{\text{tiled}} = \frac{32}{4} = 8 \text{ FLOPs per byte.}$$

This is a simplified model because it ignores output writes, boundary padding, cache behavior, and instruction overhead. But it captures the reason larger tiles can improve reuse: increasing T increases work faster than it increases global-memory traffic.

4.5 Occupancy and Resource Trade-Offs

A tile width T implies:

$$\text{threads per block} = T^2.$$

For $T = 16$:

$$T^2 = 256.$$

For $T = 32$:

$$T^2 = 1024.$$

Many NVIDIA GPUs limit the maximum number of threads per block to 1024, so $T = 32$ is often the largest square tile possible if each thread computes one output element.

The shared memory required for two FP32 tiles is:

$$S = 2T^2 \cdot 4 \text{ bytes.}$$

For $T = 16$:

$$S = 2 \cdot 16^2 \cdot 4 = 2048 \text{ bytes.}$$

For $T = 32$:

$$S = 2 \cdot 32^2 \cdot 4 = 8192 \text{ bytes.}$$

Larger tiles increase reuse but also increase per-block resource usage. Per-block resource usage affects how many blocks can reside on an SM at once. The number of resident blocks influences occupancy:

$$\text{occupancy} = \frac{\text{active warps on SM}}{\text{maximum supported active warps on SM}}.$$

High occupancy helps hide latency, but maximum occupancy is not always the same as maximum performance. For matrix multiplication, data reuse, instruction efficiency, memory coalescing, and tensor-core usage may matter more than occupancy alone.

4.6 Memory Coalescing

Memory coalescing happens when threads in a warp access contiguous memory locations. Coalesced global loads are served efficiently because the hardware combines them into fewer memory transactions.

In the tiled kernel, consider loading $M_s[t_y, t_x]$:

$$M_s[t_y, t_x] = M[r, \phi T + t_x].$$

For a fixed t_y , adjacent threads with adjacent t_x values access adjacent elements of M . This is favorable for coalescing.

For $N_s[t_y, t_x]$:

$$N_s[t_y, t_x] = N[\phi T + t_y, c].$$

For a fixed t_y , adjacent t_x values correspond to adjacent c values. Since N is row-major, this is also favorable for coalescing.

Thus, the cooperative loads can be arranged so that both matrix tiles are loaded with mostly coalesced global accesses.

4.7 Shared-Memory Bank Conflicts

Shared memory is divided into banks. If multiple threads in a warp access different addresses in the same bank during the same instruction, the access can serialize. This is called a bank conflict.

The simple tiled kernel is a teaching kernel. It demonstrates shared-memory reuse but does not fully explore bank-conflict avoidance, vectorized loads, warp-level tiling, tensor-core fragments, or register blocking. Production GEMM kernels such as cuBLAS use far more sophisticated layouts to reduce bank conflicts and maximize tensor-core throughput.

5 Mathematical Reasoning

5.1 Naive Matrix Multiplication

Given:

$$M \in \mathbb{R}^{H \times K}, \quad N \in \mathbb{R}^{K \times W},$$

the output is:

$$P \in \mathbb{R}^{H \times W}.$$

The operation is:

$$P_{r,c} = \sum_{i=0}^{K-1} M_{r,i} N_{i,c}.$$

The total number of multiply-add pairs is:

$$HWK.$$

If each multiply-add is counted as two FLOPs, the total FLOPs are:

$$2HWK.$$

For the lecture's large example:

$$H = 5120, \quad K = 256, \quad W = 5120.$$

The output has:

$$HW = 5120 \cdot 5120 = 26,214,400$$

elements.

The multiply-add count is:

$$HWK = 5120 \cdot 5120 \cdot 256 = 6,710,886,400.$$

The FLOP count is approximately:

$$2HWK = 13,421,772,800 \approx 13.4 \text{ GFLOPs}.$$

If the naive CUDA kernel takes around 6 milliseconds, the rough throughput is:

$$\frac{13.4 \times 10^9}{6 \times 10^{-3}} \approx 2.24 \times 10^{12} \text{ FLOPs/s} = 2.24 \text{ TFLOPs/s}.$$

This is far below what modern GPUs can achieve for optimized GEMM, which is the point: a teaching kernel can expose parallelism but still leave substantial performance on the table.

5.2 Tile Decomposition

Let tile width be T . The output matrix is decomposed into tiles:

$$P^{(b_y, b_x)} \in \mathbb{R}^{T \times T},$$

where:

$$b_y = \text{blockIdx.y}, \quad b_x = \text{blockIdx.x}.$$

The local thread coordinates are:

$$t_y = \text{threadIdx.y}, \quad t_x = \text{threadIdx.x}.$$

The global output coordinates are:

$$r = b_y T + t_y,$$

$$c = b_x T + t_x.$$

The phase index is:

$$\phi = 0, 1, \dots, \left\lceil \frac{K}{T} \right\rceil - 1.$$

For each phase, each thread loads:

$$M_s[t_y, t_x] = \begin{cases} M[r, \phi T + t_x], & r < H \text{ and } \phi T + t_x < K, \\ 0, & \text{otherwise,} \end{cases}$$

and:

$$N_s[t_y, t_x] = \begin{cases} N[\phi T + t_y, c], & \phi T + t_y < K \text{ and } c < W, \\ 0, & \text{otherwise.} \end{cases}$$

Then each thread accumulates:

$$\text{acc}_{r,c} \leftarrow \text{acc}_{r,c} + \sum_{j=0}^{T-1} M_s[t_y, j] N_s[j, t_x].$$

After all phases:

$$P_{r,c} = \text{acc}_{r,c}.$$

The store is guarded:

$$\text{write only if } r < H \text{ and } c < W.$$

5.3 Boundary Handling and Padding

If H , K , or W is not divisible by T , some blocks or phases extend beyond the matrix boundary. Instead of creating special cases for smaller final tiles, the kernel uses guards and zero padding.

For M :

$$M_s[t_y, t_x] = 0 \quad \text{if} \quad r \geq H \quad \text{or} \quad \phi T + t_x \geq K.$$

For N :

$$N_s[t_y, t_x] = 0 \quad \text{if} \quad \phi T + t_y \geq K \quad \text{or} \quad c \geq W.$$

This preserves correctness because multiplying by zero contributes nothing to the dot product.

5.4 Shared Memory Size

For dynamic shared memory, the kernel requests one contiguous shared-memory buffer. It then partitions the buffer into two logical arrays:

$$M_s \quad \text{and} \quad N_s.$$

Each tile contains:

$$T^2$$

floats. Two tiles contain:

$$2T^2$$

floats. Since each FP32 float is 4 bytes, the requested dynamic shared memory is:

$$S_{\text{bytes}} = 2T^2 \cdot 4.$$

In C++ code:

```
int shared_mem_size = tile_width * tile_width * 2 * sizeof(float);
```

For $T = 16$:

$$S_{\text{bytes}} = 2 \cdot 16^2 \cdot 4 = 2048.$$

For $T = 32$:

$$S_{\text{bytes}} = 2 \cdot 32^2 \cdot 4 = 8192.$$

6 Code Walkthrough

6.1 Utility: Dimension Object

The lecture uses a small Python helper similar to CUDA's `dim3`. It lets the code pass around grid and block dimensions with `x`, `y`, and `z` fields.

```

from collections import namedtuple

dim3 = namedtuple("dim3", ["x", "y", "z"], defaults=(1, 1))

```

This mirrors CUDA's launch configuration style:

```
grid dimensions = (blocks.x, blocks.y, blocks.z),
```

```
block dimensions = (threads.x, threads.y, threads.z).
```

For two-dimensional matrix multiplication, the implicit z dimension is 1.

6.2 Naive Python Matrix Multiplication Kernel

The first teaching implementation is pure Python. It simulates CUDA blocks and threads with loops.

```

def matmul_k(blockIdx, threadIdx, blockDim, m, n, out, h, k, w):
    r = blockIdx.y * blockDim.y + threadIdx.y
    c = blockIdx.x * blockDim.x + threadIdx.x

    if r >= h or c >= w:
        return

    acc = 0.0
    for i in range(k):
        acc += m[r * k + i] * n[i * w + c]

    out[r * w + c] = acc

```

The row and column are computed as:

```
r = blockIdx.y * blockDim.y + threadIdx.y,
```

```
c = blockIdx.x * blockDim.x + threadIdx.x.
```

The guard:

```

if r >= h or c >= w:
    return

```

prevents out-of-bounds writes when the grid covers a padded region beyond the real matrix shape.

The inner loop:

```

for i in range(k):
    acc += m[r * k + i] * n[i * w + c]

```

performs the dot product:

$$P_{r,c} = \sum_{i=0}^{K-1} M_{r,i} N_{i,c}.$$

In hardware terms, this naive approach repeatedly reads from global memory. Each thread independently loads the elements needed for its own dot product, even if neighboring threads need the same values.

6.3 Naive CUDA C Kernel

The equivalent CUDA C kernel has the same logic but uses CUDA’s built-in thread and block indices.

```
__global__ void matmul_k(  
    const float* m,  
    const float* n,  
    float* out,  
    int h,  
    int k,  
    int w  
) {  
    int r = blockIdx.y * blockDim.y + threadIdx.y;  
    int c = blockIdx.x * blockDim.x + threadIdx.x;  
  
    if (r >= h || c >= w) return;  
  
    float acc = 0.0f;  
    for (int i = 0; i < k; ++i) {  
        acc += m[r * k + i] * n[i * w + c];  
    }  
  
    out[r * w + c] = acc;  
}
```

The launch wrapper uses a two-dimensional grid:

```
void matmul(  
    const float* m,  
    const float* n,  
    float* out,  
    int h,  
    int k,  
    int w  
) {  
    dim3 threads(16, 16);  
    dim3 blocks((w + threads.x - 1) / threads.x,  
                (h + threads.y - 1) / threads.y);  
  
    matmul_k<<<blocks, threads>>>(m, n, out, h, k, w);  
}
```

The number of blocks in the x dimension covers output columns:

$$\text{blocks.x} = \left\lceil \frac{W}{16} \right\rceil.$$

The number of blocks in the y dimension covers output rows:

$$\text{blocks.y} = \left\lceil \frac{H}{16} \right\rceil.$$

This kernel is conceptually simple and useful for learning. It is not close to cuBLAS-level performance because it does not use shared memory tiling, register blocking, warp-level matrix multiply, vectorized loads, or tensor cores.

6.4 Python Shared-Memory Simulation

Before writing CUDA shared-memory code, the lecture constructs a Python simulation. The important trick is that tensor or array views can refer to the same underlying storage. If one view is modified, the original storage changes too.

```
import torch

a = torch.zeros(5)
b = a[:3]
c = a[3:]

b[:] = 1
c[:] = 2

print(a)
```

Conceptually, this produces:

```
tensor([1., 1., 1., 2., 2.])
```

This models dynamic shared memory: allocate one contiguous buffer and create two views into it.

For tiled matrix multiplication, the shared-memory buffer has:

$$2T^2$$

float elements. The first T^2 elements are used for M_s , and the second T^2 elements are used for N_s .

```
shared = torch.zeros(tw * tw * 2)

ms = shared[:tw * tw]
ns = shared[tw * tw:]
```

This mirrors CUDA dynamic shared memory:

```
extern __shared__ float ms[];
float* ns = ms + tile_width * tile_width;
```

The Python version is not fast. Its value is pedagogical: it lets the programmer debug indexing and tile movement before committing to CUDA C.

6.5 Tiled Python Kernel: Explicit Thread Loops

A first tiled Python version manually loops over all simulated threads twice per phase: once to load shared memory and once to compute from shared memory.

```
def matmul_shared_python(
    blockIdx,
    blockDim,
    shared,
    m,
    n,
    out,
    h,
```

```

    k,
    w,
    tw
):
    ms = shared[:tw * tw]
    ns = shared[tw * tw:]

    for ph in range((k + tw - 1) // tw):
        idx = ph * tw

        for ty in range(blockDim.y):
            for tx in range(blockDim.x):
                r = blockIdx.y * blockDim.y + ty
                c = blockIdx.x * blockDim.x + tx

                m_col = idx + tx
                n_row = idx + ty

                if r < h and m_col < k:
                    ms[ty * tw + tx] = m[r * k + m_col]
                else:
                    ms[ty * tw + tx] = 0.0

                if n_row < k and c < w:
                    ns[ty * tw + tx] = n[n_row * w + c]
                else:
                    ns[ty * tw + tx] = 0.0

            for ty in range(blockDim.y):
                for tx in range(blockDim.x):
                    r = blockIdx.y * blockDim.y + ty
                    c = blockIdx.x * blockDim.x + tx

                    if r < h and c < w:
                        acc = 0.0
                        for j in range(tw):
                            acc += ms[ty * tw + j] * ns[j * tw + tx]
                        out[r * w + c] += acc

```

This code is not exactly the final CUDA structure because real CUDA threads execute the same kernel body concurrently. But it makes the logic explicit.

The phase index:

```
idx = ph * tw
```

is:

$$\text{idx} = \phi T.$$

The load from M is:

```
ms[ty * tw + tx] = m[r * k + m_col]
```

which represents:

$$M_s[t_y, t_x] = M[r, \phi T + t_x].$$

The load from N is:

```
ns[ty * tw + tx] = n[n_row * w + c]
```

which represents:

$$N_s[t_y, t_x] = N[\phi T + t_y, c].$$

The inner dot product:

```
for j in range(tw):
    acc += ms[ty * tw + j] * ns[j * tw + tx]
```

represents:

$$\sum_{j=0}^{T-1} M_s[t_y, j] N_s[j, t_x].$$

6.6 Refactoring Toward CUDA: Run All Threads

The lecture then refactors the Python simulation to separate the idea of “run this function for every thread.” This makes the Python structure closer to CUDA.

```
def run_threads(blockDim, f):
    for ty in range(blockDim.y):
        for tx in range(blockDim.x):
            f(ty, tx)
```

Then a phase can be written as:

```
for ph in range(cdiv(k, tw)):
    idx = ph * tw

    def fill_shared(ty, tx):
        r = blockIdx.y * blockDim.y + ty
        c = blockIdx.x * blockDim.x + tx

        m_col = idx + tx
        n_row = idx + ty

        ms[ty * tw + tx] = (
            m[r * k + m_col] if r < h and m_col < k else 0.0
        )

        ns[ty * tw + tx] = (
            n[n_row * w + c] if n_row < k and c < w else 0.0
        )

    def dot_from_shared(ty, tx):
        r = blockIdx.y * blockDim.y + ty
        c = blockIdx.x * blockDim.x + tx
```

```

    if r < h and c < w:
        acc = 0.0
        for j in range(tw):
            acc += ms[ty * tw + j] * ns[j * tw + tx]
            out[r * w + c] += acc

run_threads(blockDim, fill_shared)
run_threads(blockDim, dot_from_shared)

```

This version makes the conceptual CUDA program visible:

1. For every phase, all threads fill shared memory.
2. Then all threads compute using shared memory.

However, real CUDA does not run two explicit loops over threads. Instead, every thread runs the same kernel body, and barriers coordinate the phases.

6.7 Modeling CUDA Synchronization with Python Threads

The lecture introduces Python threads and barriers to simulate CUDA's concurrent execution model. A barrier forces all threads to reach a point before any thread continues.

A simple demonstration:

```

from concurrent.futures import ThreadPoolExecutor
from threading import Barrier

def g(i, sync):
    print(i)
    sync.wait()
    print(-i)
    sync.wait()
    print(i * 10)

num_threads = 3
sync = Barrier(num_threads)

with ThreadPoolExecutor(max_workers=num_threads) as ex:
    ex.map(lambda i: g(i, sync), range(num_threads))

```

The important concept is not Python performance. The point is that a barrier creates phases:

all threads finish step 1 → all threads finish step 2 → all threads finish step 3.

CUDA's equivalent is:

```
__syncthreads();
```

6.8 Python Kernel Runner with Shared Memory and Barriers

A more CUDA-like Python runner creates a shared buffer and a synchronization barrier for each block.

```

from concurrent.futures import ThreadPoolExecutor
from threading import Barrier
import torch

def shared_kernel_runner(blocks, threads, shared_size, kernel, *args):
    for by in range(blocks.y):
        for bx in range(blocks.x):
            blockIdx = dim3(bx, by)
            blockDim = threads
            shared = torch.zeros(shared_size)
            sync = Barrier(threads.x * threads.y)

            jobs = []
            with ThreadPoolExecutor(max_workers=threads.x * threads.y) as ex:
                for ty in range(threads.y):
                    for tx in range(threads.x):
                        threadIdx = dim3(tx, ty)
                        jobs.append(
                            ex.submit(
                                kernel,
                                blockIdx,
                                threadIdx,
                                blockDim,
                                shared,
                                sync,
                                *args
                            )
                        )

            for job in jobs:
                job.result()

```

This runner passes each simulated thread:

- its block index,
- its thread index,
- block dimensions,
- the shared-memory buffer,
- the synchronization barrier,
- the kernel arguments.

This is close to the CUDA model, except that CUDA supplies `blockIdx`, `threadIdx`, `blockDim`, and `__syncthreads()` implicitly.

6.9 CUDA-Like Tiled Kernel in Python

The CUDA-like Python kernel computes one output element per thread and uses a barrier to separate shared-memory loading from shared-memory computation.

```

def matmul_tiled_python_kernel(
    blockIdx,
    threadIdx,
    blockDim,
    shared,
    sync,
    m,
    n,
    out,
    h,
    k,
    w,
    tw
):
    tx = threadIdx.x
    ty = threadIdx.y

    r = blockIdx.y * blockDim.y + ty
    c = blockIdx.x * blockDim.x + tx

    ms = shared[:tw * tw]
    ns = shared[tw * tw:]

    acc = 0.0

    for ph in range((k + tw - 1) // tw):
        idx = ph * tw

        m_col = idx + tx
        n_row = idx + ty

        if r < h and m_col < k:
            ms[ty * tw + tx] = m[r * k + m_col]
        else:
            ms[ty * tw + tx] = 0.0

        if n_row < k and c < w:
            ns[ty * tw + tx] = n[n_row * w + c]
        else:
            ns[ty * tw + tx] = 0.0

        sync.wait()

        for j in range(tw):
            acc += ms[ty * tw + j] * ns[j * tw + tx]

        sync.wait()

    if r < h and c < w:
        out[r * w + c] = acc

```

The first barrier:

```
sync.wait()
```

ensures the shared-memory tiles are fully loaded.

The second barrier:

```
sync.wait();
```

ensures no thread overwrites shared memory for the next phase while another thread is still using it.

This is the Python equivalent of:

```
__syncthreads();
```

6.10 CUDA C Kernel with Dynamic Shared Memory

The CUDA C version uses dynamic shared memory through the special declaration:

```
extern __shared__ float ms[];
```

This gives a pointer to a runtime-sized shared-memory buffer. The second tile is placed after the first tile:

```
float* ns = ms + tile_width * tile_width;
```

A representative dynamic shared-memory tiled kernel is:

```
__global__ void matmul_tiled_dynamic_k(  
    const float* m,  
    const float* n,  
    float* out,  
    int h,  
    int k,  
    int w,  
    int tile_width  
) {  
    int tx = threadIdx.x;  
    int ty = threadIdx.y;  
  
    int r = blockIdx.y * blockDim.y + ty;  
    int c = blockIdx.x * blockDim.x + tx;  
  
    extern __shared__ float ms[];  
    float* ns = ms + tile_width * tile_width;  
  
    float acc = 0.0f;  
  
    int phases = (k + tile_width - 1) / tile_width;  
  
    for (int ph = 0; ph < phases; ++ph) {  
        int idx = ph * tile_width;  
  
        int m_col = idx + tx;  
        int n_row = idx + ty;  
  
        if (r < h && m_col < k) {  
            ms[ty * tile_width + tx] = m[r * k + m_col];  
        } else {
```



```

        ms[ty * tile_width + tx] = 0.0f;
    }

    if (n_row < k && c < w) {
        ns[ty * tile_width + tx] = n[n_row * w + c];
    } else {
        ns[ty * tile_width + tx] = 0.0f;
    }

    __syncthreads();

    for (int j = 0; j < tile_width; ++j) {
        acc += ms[ty * tile_width + j] *
              ns[j * tile_width + tx];
    }

    __syncthreads();
}

if (r < h && c < w) {
    out[r * w + c] = acc;
}
}

```

The launch passes dynamic shared-memory size as the third execution-configuration argument:

```

void matmul_tiled_dynamic(
    const float* m,
    const float* n,
    float* out,
    int h,
    int k,
    int w
) {
    int tile_width = 16;

    dim3 threads(tile_width, tile_width);
    dim3 blocks((w + tile_width - 1) / tile_width,
                (h + tile_width - 1) / tile_width);

    int shared_mem_size =
        tile_width * tile_width * 2 * sizeof(float);

    matmul_tiled_dynamic_k<<<blocks, threads, shared_mem_size>>>(
        m, n, out, h, k, w, tile_width
    );
}

```

The third launch parameter:

```
shared_mem_size
```

tells CUDA how many bytes of dynamic shared memory to allocate per block.

6.11 Static Shared Memory

If the tile width is known at compile time, shared memory can be declared statically:

```
constexpr int TILE_WIDTH = 16;

__shared__ float ms[TILE_WIDTH][TILE_WIDTH];
__shared__ float ns[TILE_WIDTH][TILE_WIDTH];
```

A representative static version is:

```
constexpr int TILE_WIDTH = 16;

__global__ void matmul_tiled_static_k(
    const float* m,
    const float* n,
    float* out,
    int h,
    int k,
    int w
) {
    int tx = threadIdx.x;
    int ty = threadIdx.y;

    int r = blockIdx.y * TILE_WIDTH + ty;
    int c = blockIdx.x * TILE_WIDTH + tx;

    __shared__ float ms[TILE_WIDTH][TILE_WIDTH];
    __shared__ float ns[TILE_WIDTH][TILE_WIDTH];

    float acc = 0.0f;

    int phases = (k + TILE_WIDTH - 1) / TILE_WIDTH;

    for (int ph = 0; ph < phases; ++ph) {
        int idx = ph * TILE_WIDTH;

        int m_col = idx + tx;
        int n_row = idx + ty;

        if (r < h && m_col < k) {
            ms[ty][tx] = m[r * k + m_col];
        } else {
            ms[ty][tx] = 0.0f;
        }

        if (n_row < k && c < w) {
            ns[ty][tx] = n[n_row * w + c];
        } else {
            ns[ty][tx] = 0.0f;
        }

        __syncthreads();

        for (int j = 0; j < TILE_WIDTH; ++j) {
```

```

        acc += ms[ty][j] * ns[j][tx];
    }

    __syncthreads();
}

if (r < h && c < w) {
    out[r * w + c] = acc;
}
}

```

The static version has cleaner indexing:

```

ms[ty][tx]
ns[ty][tx]

```

instead of flattened shared-memory indexing:

```

ms[ty * tile_width + tx]
ns[ty * tile_width + tx]

```

More importantly, if `TILE_WIDTH` is a compile-time constant, the compiler can specialize and optimize the loop over j more aggressively. It may unroll loops, simplify address arithmetic, and generate better machine code.

6.12 Why Dynamic Tile Width Can Be Slower

The lecture initially observes a surprising result: the dynamic shared-memory version can be slower than the naive kernel or slower than the static shared-memory version. The later explanation is that the dynamic version passed `tile_width` as a runtime value. When tile width is not known at compile time, the compiler cannot specialize the loop and indexing for a fixed size.

The critical issue is not merely dynamic shared-memory allocation itself. The issue is that this loop:

```

for (int j = 0; j < tile_width; ++j) {
    acc += ms[ty * tile_width + j] *
           ns[j * tile_width + tx];
}

```

has runtime-dependent bounds and runtime-dependent address strides.

If `tile_width` is known at compile time, the compiler can reason about:

$$j = 0, 1, \dots, T - 1,$$

where T is fixed. With runtime tile width, it must generate more general code. This can inhibit optimization.

6.13 Template-Based Tile Width

A better approach is to make tile width a C++ template parameter:

```

template <int TILE_WIDTH>
__global__ void matmul_tiled_template_k(
    const float* m,
    const float* n,

```

```

    float* out,
    int h,
    int k,
    int w
) {
    int tx = threadIdx.x;
    int ty = threadIdx.y;

    int r = blockIdx.y * TILE_WIDTH + ty;
    int c = blockIdx.x * TILE_WIDTH + tx;

    __shared__ float ms[TILE_WIDTH][TILE_WIDTH];
    __shared__ float ns[TILE_WIDTH][TILE_WIDTH];

    float acc = 0.0f;
    int phases = (k + TILE_WIDTH - 1) / TILE_WIDTH;

    for (int ph = 0; ph < phases; ++ph) {
        int idx = ph * TILE_WIDTH;

        int m_col = idx + tx;
        int n_row = idx + ty;

        ms[ty][tx] = (r < h && m_col < k)
            ? m[r * k + m_col]
            : 0.0f;

        ns[ty][tx] = (n_row < k && c < w)
            ? n[n_row * w + c]
            : 0.0f;

        __syncthreads();

        #pragma unroll
        for (int j = 0; j < TILE_WIDTH; ++j) {
            acc += ms[ty][j] * ns[j][tx];
        }

        __syncthreads();
    }

    if (r < h && c < w) {
        out[r * w + c] = acc;
    }
}

```

Then the host code can choose among a few supported tile widths:

```

void matmul_tiled_template(
    const float* m,
    const float* n,
    float* out,
    int h,
    int k,
    int w,

```

```

    int tile_width
) {
    auto launch16 = [&]() {
        dim3 threads(16, 16);
        dim3 blocks((w + 15) / 16, (h + 15) / 16);
        matmul_tiled_template_k<16><<<blocks, threads>>>(
            m, n, out, h, k, w
        );
    };

    auto launch32 = [&]() {
        dim3 threads(32, 32);
        dim3 blocks((w + 31) / 32, (h + 31) / 32);
        matmul_tiled_template_k<32><<<blocks, threads>>>(
            m, n, out, h, k, w
        );
    };

    if (tile_width == 16) {
        launch16();
    } else if (tile_width == 32) {
        launch32();
    }
}

```

This pattern gives a runtime choice among several compile-time-specialized kernels. It is less elegant than passing any tile width dynamically, but it is much more compatible with compiler optimization.

6.14 Querying Device Properties

The lecture briefly mentions that CUDA exposes device properties such as maximum threads per block and shared memory per block. A representative pattern is:

```

cudaDeviceProp prop;
cudaGetDeviceProperties(&prop, 0);

int max_threads_per_block = prop.maxThreadsPerBlock;
size_t shared_mem_per_block = prop.sharedMemPerBlock;

```

These properties can guide launch-configuration choices. For example, a tile width must satisfy:

$$T^2 \leq \text{maxThreadsPerBlock}.$$

The shared-memory allocation must satisfy:

$$2T^2 \cdot 4 \leq \text{sharedMemPerBlock}.$$

In practice, choosing an optimal tile size also depends on register usage, occupancy, matrix shape, memory coalescing, warp-level tiling, and whether tensor cores are used.

7 Numba CUDA Development Path

7.1 Why Numba Appears in the Lecture

The lecture introduces Numba as another way to write CUDA kernels from Python. Instead of writing CUDA C strings and compiling them with PyTorch's inline extension mechanism, Numba lets the programmer write GPU kernels in Python syntax.

The key trade-off is:

- CUDA C gives direct access to the native CUDA programming model and deployment paths.
- Numba gives faster iteration, easier Python-side debugging, and more convenient indexing.

Numba is not the same as ordinary Python execution. A decorated Numba CUDA function is compiled into GPU code.

7.2 Representative Numba Tiled Matrix Multiplication

A representative Numba version looks like this:

```
from numba import cuda, float32

@cuda.jit
def matmul_tiled_numba(m, n, out):
    tw = 16

    tx = cuda.threadIdx.x
    ty = cuda.threadIdx.y

    bx = cuda.blockIdx.x
    by = cuda.blockIdx.y

    r = by * cuda.blockDim.y + ty
    c = bx * cuda.blockDim.x + tx

    h = m.shape[0]
    k = m.shape[1]
    w = n.shape[1]

    ms = cuda.shared.array((16, 16), dtype=float32)
    ns = cuda.shared.array((16, 16), dtype=float32)

    acc = 0.0

    phases = (k + tw - 1) // tw

    for ph in range(phases):
        idx = ph * tw

        m_col = idx + tx
        n_row = idx + ty

        if r < h and m_col < k:
            ms[ty, tx] = m[r, m_col]
```

```

    else:
        ms[ty, tx] = 0.0

    if n_row < k and c < w:
        ns[ty, tx] = n[n_row, c]
    else:
        ns[ty, tx] = 0.0

    cuda.syncthreads()

    for j in range(tw):
        acc += ms[ty, j] * ns[j, tx]

    cuda.syncthreads()

    if r < h and c < w:
        out[r, c] = acc

```

Several details are more convenient than CUDA C:

- `m.shape`, `n.shape`, and `out.shape` are available.
- The tensors need not be manually flattened in the kernel.
- Shared memory can use two-dimensional indexing.
- Compilation is often faster for development iteration.

The CUDA concepts remain the same:

```

cuda.blockIdx ↔ blockIdx,

cuda.threadIdx ↔ threadIdx,

cuda.syncthreads() ↔ __syncthreads().

```

7.3 Dynamic Shared Memory in Numba

Numba also supports dynamic shared memory. A representative pattern is:

```

@cuda.jit
def matmul_tiled_numba_dynamic(m, n, out, tw):
    tx = cuda.threadIdx.x
    ty = cuda.threadIdx.y

    bx = cuda.blockIdx.x
    by = cuda.blockIdx.y

    r = by * cuda.blockDim.y + ty
    c = bx * cuda.blockDim.x + tx

    h = m.shape[0]
    k = m.shape[1]

```

```

w = n.shape[1]

shared = cuda.shared.array(0, dtype=float32)

ms = shared[:tw * tw]
ns = shared[tw * tw:]

acc = 0.0

for ph in range((k + tw - 1) // tw):
    idx = ph * tw

    m_col = idx + tx
    n_row = idx + ty

    if r < h and m_col < k:
        ms[ty * tw + tx] = m[r, m_col]
    else:
        ms[ty * tw + tx] = 0.0

    if n_row < k and c < w:
        ns[ty * tw + tx] = n[n_row, c]
    else:
        ns[ty * tw + tx] = 0.0

    cuda.syncthreads()

    for j in range(tw):
        acc += ms[ty * tw + j] * ns[j * tw + tx]

    cuda.syncthreads()

if r < h and c < w:
    out[r, c] = acc

```

The launch uses square brackets rather than CUDA C triple angle brackets:

```

tw = 16
threads = (tw, tw)
blocks = (cdiv(w, tw), cdiv(h, tw))
shared_mem_size = tw * tw * 2 * 4

matmul_tiled_numba_dynamic[blocks, threads, 0, shared_mem_size](
    m_cuda,
    n_cuda,
    out_cuda,
    tw
)

```

The launch format is:

```
kernel[blocks, threads, stream, sharedmem](args).
```

The fourth launch component specifies dynamic shared-memory bytes.

7.4 Numba CUDA Simulator

One of the lecture's most practical points is that Numba provides a CUDA simulator. By setting an environment variable before importing Numba:

```
import os
os.environ["NUMBA_ENABLE_CUDASIM"] = "1"
```

the same CUDA kernel can run on the CPU as simulated CUDA. This allows:

- Python breakpoints,
- ordinary print debugging,
- stepping through the kernel,
- testing with small inputs,
- debugging indexing mistakes before running on the GPU.

This is valuable because CUDA bugs are often indexing or synchronization bugs. The simulator gives a tight debugging loop.

A practical workflow is:

1. Write the kernel in Numba CUDA.
2. Enable CUDA simulation.
3. Test on tiny matrices.
4. Print shared-memory contents and thread coordinates.
5. Verify correctness against PyTorch.
6. Disable simulation.
7. Run on the GPU.
8. Benchmark.
9. Optionally convert to CUDA C for deployment or further optimization.

8 Performance Intuition

8.1 Why the Naive Kernel Can Still Be Reasonably Fast

The naive CUDA matrix multiplication can run much faster than pure Python because it exposes massive thread-level parallelism. Every output element can be computed independently, so the grid contains many threads. For a 5120×5120 output, there are over 26 million output elements, and therefore over 26 million independent thread computations.

However, the naive kernel is still inefficient by GPU standards because it relies heavily on global memory. It does not reuse data through shared memory or registers beyond one thread's local accumulator.

8.2 Why Shared Memory Does Not Automatically Win

The lecture observes that a shared-memory kernel may not always be faster than the naive version. This is an important lesson. Adding shared memory also adds:

- extra instructions to load shared memory,
- two synchronizations per tile phase,
- more complicated indexing,
- shared-memory resource usage,
- possible bank conflicts,
- compiler optimization sensitivity.

A shared-memory kernel wins only if the saved global-memory traffic outweighs these costs. For matrix multiplication, well-optimized shared-memory tiling is essential, but a first teaching implementation may still be far from optimal.

8.3 Dynamic Versus Static Shared Memory

The surprising performance result in the lecture is that dynamic shared memory with runtime tile width can be slower than static shared memory. The core lesson is:

The compiler can optimize fixed-size loops and fixed-stride address arithmetic much better than fully dynamic loops and dynamic indexing.

With a compile-time tile width:

$$T = 16,$$

the compiler knows the loop trip count:

$$j = 0, \dots, 15.$$

It can unroll the loop and simplify indexing. With runtime tile width, the compiler must support many possible values of T , so it may generate more conservative code.

8.4 Why Production GEMM Is Much Harder

The lecture's tiled kernel is a first serious step, not a production GEMM. Libraries such as cuBLAS and cuDNN use many additional optimizations:

- multiple output elements per thread,
- register tiling,
- warp-level tiling,
- tensor-core instructions,
- vectorized memory loads,

- asynchronous copies from global memory to shared memory,
- double buffering,
- carefully chosen shared-memory layouts,
- bank-conflict avoidance,
- architecture-specific tuning,
- autotuning across matrix shapes.

The simple kernel computes one output element per thread. Modern high-performance GEMM kernels usually compute multiple output elements per thread or per warp and use specialized matrix multiply-accumulate instructions.

8.5 Shape Sensitivity

The lecture notes that the original previous example used a long narrow matrix shape related to MNIST. Such shapes can be harder to optimize with square tiles. For example, if the output has only 10 columns and the tile width is 32, most of each output tile is padding:

$$\frac{10}{32}$$

of the tile columns are useful, and the rest are out of bounds. This wastes threads and shared-memory work.

For square-ish matrices such as:

$$5120 \times 256 \quad \text{multiplied by} \quad 256 \times 5120,$$

square tiles are easier to reason about and use more of the launched threads.

8.6 Kernel Launch Overhead

The lecture compares the time spent doing matrix multiplication with overhead outside the kernel. By commenting out the heavy matrix multiplication and timing the surrounding work, the overhead is much smaller than the total runtime. This is a useful benchmarking discipline:

Do not assume a measured time is kernel compute time. Isolate kernel launch, setup, synchronization, and memory allocation overhead when interpreting benchmarks.

For tiny workloads, launch overhead can dominate. For large matrix multiplication, the kernel runtime usually dominates.

8.7 Compilation Speed and Iteration Loop

CUDA C extensions can have slow compile times. PyTorch inline extension compilation is convenient but still uses the C++ and CUDA toolchain. Numba compilation can be much faster for iterative development. This matters because kernel development often involves many cycles of:

1. edit,

2. compile,
3. run,
4. inspect wrong result,
5. debug indexing,
6. repeat.

A fast iteration loop is not just convenience. It changes how quickly one can learn and experiment.

9 Common Misconceptions

9.1 Misconception: Shared Memory Is Global Across the GPU

Shared memory is not global. It is local to a block. Threads in one block cannot read another block's shared memory.

Correct mental model:

one block $\rightarrow$ one shared-memory region.

9.2 Misconception: A CUDA Block and a Tile Are the Same Thing

A block is a CUDA execution object. A tile is an algorithmic region of a matrix. In this lecture, we choose to map one block to one output tile, but that is a design decision.

Correct mental model:

block = hardware/programming cooperation group,

tile = algorithmic data region.

9.3 Misconception: Adding Shared Memory Automatically Makes a Kernel Faster

Shared memory helps only when it increases reuse enough to overcome its overhead. A poorly written shared-memory kernel can be slower than a naive kernel.

Reasons include:

- synchronization overhead,
- bad memory coalescing,
- bank conflicts,
- low occupancy,
- poor compiler optimization,
- unnecessary shared-memory loads,
- tile sizes that do not match the shape.

9.4 Misconception: Dynamic Shared Memory Is Always Equivalent to Static Shared Memory

Dynamic shared memory can be equivalent in principle, but performance may differ if important constants become runtime variables. The compiler cannot optimize runtime tile width as aggressively as compile-time tile width.

Correct mental model:

dynamic allocation $\neq$ dynamic tile width should be free.

The expensive part may be the loss of compile-time specialization.

9.5 Misconception: Learning Triton Means You Can Skip CUDA Concepts

Triton is higher-level and powerful, but it does not remove the need to understand memory hierarchy, tiling, coalescing, synchronization, and hardware execution. Triton expresses GPU programs differently, but it still targets the same hardware realities.

Correct mental model:

CUDA teaches the hardware model. Triton gives another programming interface. Effective Triton still requires CUDA-like performance intuition.

9.6 Misconception: Python CUDA Means Ordinary Python Runs on the GPU

Numba CUDA uses Python syntax, but the kernel is compiled to GPU code. Not all Python features are supported inside kernels. The programmer still writes in the CUDA programming model, using thread indices, blocks, shared memory, and synchronization.

10 Practical Takeaways

10.1 Takeaway 1: Draw the Indexing

The lecture strongly emphasizes drawing the tiles and labeling indices. Matrix multiplication tiling is easy to get wrong if one tries to reason only from code.

For each tile, draw:

- the output tile,
- the corresponding row block of M ,
- the corresponding column block of N ,
- the phase index ϕ ,
- local thread coordinates (t_y, t_x) ,
- global coordinates (r, c) .

10.2 Takeaway 2: Think in Phases

A tiled matrix multiplication kernel is a loop over phases. Each phase has a repeated structure:

load $\rightarrow$ synchronize $\rightarrow$ compute $\rightarrow$ synchronize.

This phase structure is the backbone of the kernel.

10.3 Takeaway 3: Shared Memory Is Programmer-Managed Cache

Shared memory acts like a manually controlled cache. The programmer decides:

- what data to load,
- when to load it,
- how threads cooperate,
- when to synchronize,
- when it is safe to overwrite the buffer.

Unlike ordinary cache, shared memory is explicit. This makes it powerful but also error-prone.

10.4 Takeaway 4: Compile-Time Constants Matter

When optimizing CUDA kernels, values such as tile width are often better as compile-time constants. Templates are a common way to preserve specialization while supporting multiple tile sizes.

A common pattern is:

runtime choice $\rightarrow$ dispatch to compile-time-specialized kernel.

10.5 Takeaway 5: Use Numba for Learning and Debugging

Numba CUDA is useful because it provides:

- faster iteration,
- Python syntax,
- easier shape access,
- CUDA simulation,
- direct print and breakpoint debugging.

A strong workflow is to prototype in Numba, validate with the simulator, benchmark on GPU, and then decide whether to keep the Numba version or convert to CUDA C.

11 Project or Experiment Ideas

11.1 Experiment 1: Naive Versus Tiled Matrix Multiplication

Implement two kernels:

1. naive one-thread-per-output matrix multiplication,
2. tiled shared-memory matrix multiplication.

Benchmark shapes:

$$512 \times 512, \quad 1024 \times 1024, \quad 2048 \times 2048.$$

Measure:

- runtime,
- effective TFLOPs,
- speedup,
- correctness against `torch.matmul`.

Effective throughput is:

$$\text{TFLOPs/s} = \frac{2HWK}{t \cdot 10^{12}},$$

where t is runtime in seconds.

11.2 Experiment 2: Tile Width Sweep

Benchmark tile widths:

$$T \in \{8, 16, 32\}.$$

For each T , record:

- threads per block,
- shared memory per block,
- runtime,
- achieved TFLOPs/s,
- correctness.

Threads per block:

$$T^2.$$

Shared memory per block:

$$2T^2 \cdot 4 \text{ bytes}.$$

This experiment teaches the trade-off between reuse, occupancy, and compiler behavior.

11.3 Experiment 3: Dynamic Versus Static Shared Memory

Create three versions:

1. dynamic shared memory with runtime tile width,
2. static shared memory with fixed tile width,
3. template-specialized tile width.

Compare performance and inspect generated code if possible. The goal is to verify the lecture's lesson that compile-time constants can significantly affect optimization.

11.4 Experiment 4: Numba Simulator Debugging

Write a Numba tiled matrix multiplication kernel and enable:

`NUMBA_ENABLE_CUDASIM=1`

Use a tiny example:

$$M \in \mathbb{R}^{3 \times 5}, \quad N \in \mathbb{R}^{5 \times 4}.$$

Use tile width:

$$T = 2.$$

Print:

- block index,
- thread index,
- phase index,
- loaded M_s value,
- loaded N_s value,
- accumulator updates.

This makes boundary padding and synchronization intuitive.

11.5 Experiment 5: Shape Sensitivity

Compare square and skinny matrix shapes:

$$1024 \times 1024 \quad \text{by} \quad 1024 \times 1024,$$

versus:

$$50000 \times 784 \quad \text{by} \quad 784 \times 10.$$

Observe how square tiles waste work on skinny outputs. Try tile shapes such as:

$$16 \times 16, \quad 16 \times 8, \quad 8 \times 32.$$

This leads naturally toward more advanced GEMM kernel design.

11.6 Experiment 6: Compare Against PyTorch

Benchmark:

```
torch.matmul(m, n)
```

against your custom kernels. Use PyTorch profiling or Nsight Systems to identify whether PyTorch calls cuBLAS. The expected result is that cuBLAS is much faster for standard GEMM shapes.

The purpose is not to beat cuBLAS immediately. The purpose is to understand what optimizations remain.

11.7 Experiment 7: Use Nsight Compute

Profile the tiled kernel with Nsight Compute. Look at:

- achieved occupancy,
- global memory load efficiency,
- shared-memory throughput,
- warp stall reasons,
- memory coalescing,
- register usage,
- shared-memory bank conflicts.

Connect profiler metrics to source code. For example, if memory stalls dominate, inspect global load patterns. If synchronization stalls dominate, inspect the number of phases and barriers.

12 Final Mental Model

The lecture's central mental model is:

A GPU is not fast merely because it has many threads. It is fast when many threads cooperate to reuse data close to the compute units. For matrix multiplication, each block should compute an output tile. The block repeatedly loads one tile of M and one tile of N from global memory into shared memory, synchronizes, performs many dot-product updates from shared memory, synchronizes again, and moves to the next phase. Shared memory turns redundant global-memory reads into fast on-chip reuse.

In formula form, the transition is:

$$\text{naive : } P_{r,c} = \sum_{i=0}^{K-1} M_{r,i} N_{i,c} \quad \text{with repeated global loads,}$$

to:

$$\text{tiled : } P_{r,c} = \sum_{\phi=0}^{\lceil K/T \rceil - 1} \sum_{j=0}^{T-1} M_s[t_y, j] N_s[j, t_x],$$

where:

$$M_s[t_y, t_x] = M[r, \phi T + t_x],$$

and:

$$N_s[t_y, t_x] = N[\phi T + t_y, c].$$

The programming pattern is:

load tile $\rightarrow$ synchronize $\rightarrow$ compute from shared memory $\rightarrow$ synchronize $\rightarrow$ next tile

The engineering lesson is that CUDA performance lives at the intersection of algorithmic tiling, memory hierarchy, compiler specialization, synchronization, occupancy, and hardware-aware benchmarking. Numba can make the learning and debugging loop faster, but the underlying concepts remain CUDA concepts: blocks, threads, warps, shared memory, global memory, synchronization, coalescing, occupancy, and arithmetic intensity.